Spotify tracks: A Multitask Machine Learning Study

Angelo Zardini
University of Verona
Verona, Italy
angelo.zardini@studenti.univr.it

Abstract

The rapid growth of music streaming over the past decade, largely replacing physical media such as CDs and vinyl, has changed the way listeners interact with music. Music streaming platforms generate a huge amount of data about listener behavior, increasingly used to provide recommendations and personalize the user experience. This project adopts a multitask machine learning approach on a public Spotify Dataset, combining regression, classification and clustering to extract meaningful patterns from a song's acoustic features. Specifically, regression is used to predict track popularity, classification to determine musical genre, and clustering to group tracks into broader categories, eventually supporting a content-based recommendation system. Results show that audio features alone carry limited predictive power for popularity ($R^2 \approx 0.41$), while crafting external features, like artist and genre identity, highly improves performance ($R^2 \approx 0.55$, test RMSE $\approx$ 12.98). For genre classification, native labels proved to be too acoustically overlapping to predict directly ($F_{1,macro} \approx 0.07$); grouping them into five macro-genres via clustering made the classes far more separable ($F_{1,macro} \approx 0.79$). Finally, these results were reused to build a content-based recommender, which avoids popularity bias and introduces an alternative discovery mode based on the regression model's residuals.

1. Introduction

The project uses the publicly available dataset uploaded on hugging face "spotify tracks dataset"[2]. The data is collected using Spotify's Web API and Python, and will be used for:

- Regression based on track popularity
- Classification purposes based on audio features and available genres
- Building a recommendation system based on some user input or preference

The remainder of this report is organized as follows: Section 2 shows the details of the preprocessing pipeline and the methodology for each task; Section 3 presents the results of these tasks; Section 4 concludes discussing the main findings and limitations.

1.1. Dataset Description

The dataset contains 114000 tracks over a range of 114 different genres. Each track has some audio features associated to it. The data is in CSV format and can be loaded quickly using a Pandas dataframe. It contains 20 columns, we can divide them in identifier features, audio features (continuous and discrete) and labels. These 3 sets are shown respectively in the three tables below.

Table 1. ID Features.

Feature Name	Description
track_id	*The spotify ID for the track.*
artists	*The artists' names who performed the track.*
album_name	*The album name in which the track appears.*
track_name	*The name of the track.*

Table 2. Targets.

Target Name	Description
popularity	*Popularity score with a 0-100 range, based on the total number of plays the track has had and how recent those plays are.*
track_genre	*The genre in which the track belongs.*

For the regression task, the `popularity` column is used as a target, while for the classification task the target

Table 3. Audio Features.

Feature Name	Description
duration_ms	*The track length in milliseconds.*
explicit	*Whether or not the track has explicit lyrics.*
danceability	*How suitable a track is for dancing based on a combination of musical elements including tempo, rhythm stability, beat strength, and overall regularity.*
energy	*A measure from 0.0 to 1.0 and represents a perceptual measure of intensity and activity.*
key	*The key the track is in using standard Pitch Class notation (e.g. 0 = C, 1 = C ♯ / D ♭, 2 = D, etc).*
loudness	*The overall loudness of a track in decibels (dB).*
mode	*The modality (major or minor) of a track.*
speechiness	*Ranged from 0.0 to 1.0, detects the presence of spoken words in a track.*
acousticness	*A confidence measure from 0.0 to 1.0 of whether the track is acoustic.*
instrumentalness	*Predicts whether a track contains no vocals.*
liveness	*Detects the presence of an audience in the recording.*
valence	*A measure from 0.0 to 1.0 describing the musical positiveness conveyed by a track.*
tempo	*The overall estimated tempo of a track in beats per minute (BPM).*
time_signature	*An estimated time signature (from 3/4 to 7/4).*

is `track_genre`. Since the same song can have multiple genres (a track can appear more than one time, but labeled differently, e.g.ambient and chill) and similar genre are closer in the feature space, another approach based on clustering of the music genres to give more relaxed labels is explored.

1.2. Objective

Since the project is divided in three tasks, the main objectives can be described as follows:

- *Popularity Regression*: quantify how much the single native audio features can explain and predict the index of popularity, a continuous value in the range [0, 100], also evaluating the contribution of aggregate feature deriving from artists in order to mitigate the high variance of the dataset.
- *Macro-genre Classification*: solve the ambiguity of the 114 native genres and their acoustic overlapping, evaluating classes of classification models on some macro-genre derived by an unsupervised clustering of the centroids.
- *Content-Based Music Recommendation*: exploit the acoustic representation obtained from the 2 previous task to implement a recommendation system that suggests similar tracks starting by an user input query.

1.3. Related Works

The landscape of Music Information Retrieval (MIR) and recommendation systems has evolved significantly, shifting from traditional collaborative filtering to hybrid models that incorporate content-based analysis.

Recommender systems historically relied on Matrix Factorization and user-item interaction matrices [1]. While these methods are effective at capturing users' preferences, they are highly limited by a "cold-start" problem, struggling to provide accurate recommendations for new tracks or users with limited interaction history. To overcome this problem, content-based approaches leverage acoustic descriptors directly. However, as highlighted in foundational MIR surveys [3], bridging the semantic gap between acoustic signals and more high-level concepts, such as genres or emotional valence, remains a major struggle due to the subjective nature of music categorization.

Predicting commercial performance, formalized as *Hit Song Science* (HSS), presents some obstacles. Prior literature demostrares that acoustic features alone yield limited predictive power without artist context, as success and virality depend heavily on marketing campaigns and social network interactions. Similarly, fine-grained genre classification across a broad set of small genres and subgeneres, sufferes from label ambiguity and acoustic overlap, motivating the recent worls to adopt a hierarchical or macro-genre aggregations.

The approach of this project addresses these limitations by adopting a multitask framework to grasp all of these MIR dimensions. Rather than treating track metadata separately, it will be examined the ineraction between audio characteristics, such as such as danceability, energy, and acousticness, combined with commercial popularity and acoustic genre affinities.

By performing regression for popularity prediction, clas-

sification for genre identification, and clustering for content grouping all togheter, the model learns a more robust feature representation. Combining these tasks not only improves the predictive performance of each individual models but also helps the recommender make better suggestions, balancing objective audio features with observed music trends.

1.4. Exploratory Data Analysis

Before training the models, a strong exploratory analysis on the dataset is performed, in order to clean, preprocess and prepare the features properly.
The data shows only one *NaN* value, which was removed. All the 114 music genres are evenly distributed (1000 tracks each in the raw data), and the dataset contains 31437 unique artists.
While displaying the *Top10* most popular songs, the data clearly presented some duplicates. Checking the pair `track_name`/`artists` identified approximately 49k duplicate rows, compared to 40.9k duplicates identified via `track_id` alone. This gap is due to two main reasons: (1) the same `track_id` is repeated under multiple genre tags and (2) distinct `track_id` values shares the same `track_name`/`artists` pair, typically corrresponding to live or remastered version with differrent popularity scores. The logic for handling the duplicates in both cases is the following: a single deduplication step is applied on the (`track_name`, `artists`) pair, keeping the row with the highest popularity and breaking ties by genre (alphabetical order) for reproducibility. This choice introduces a small negligible upward bias in the resulting popularity distribution

The distribution of the `popularity` target is shown in Figure 1.

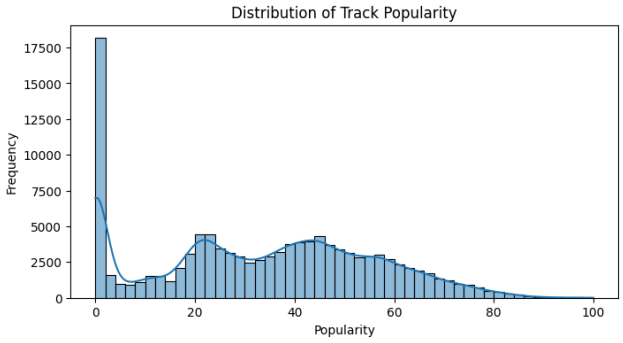

Figure 1. Distribution of track popularity across the deduplicated dataset.

It is clear that the majority of the tracks have zero popularity, this can be explained by unknown tracks of unknown artists, since Spotify allows potentially everyone to upload his music for a small fee. This motivated an initial hypothesis: predicting zero vs. non-zero popularity as a separate binary classification stage before regression (a "hurdle" architecture). As discussed in Section 3.1, this approach was tested but eventually dropped, because it showed no clear advantage over a single stage regressor.

The next step was analyzing the distribution and the correlation of the audio features, expected to be the main driver for a track popularity alongside with artist identity. Computing an artist's mean popularity directly from its own tracks would introduce data leakage, this was addresses via target encoding with K-fold cross-fitting and shrinkage toward the global mean, as discussed later in Section 3.1

The correlation matrix of audio features is shown in Figure 2. 2:

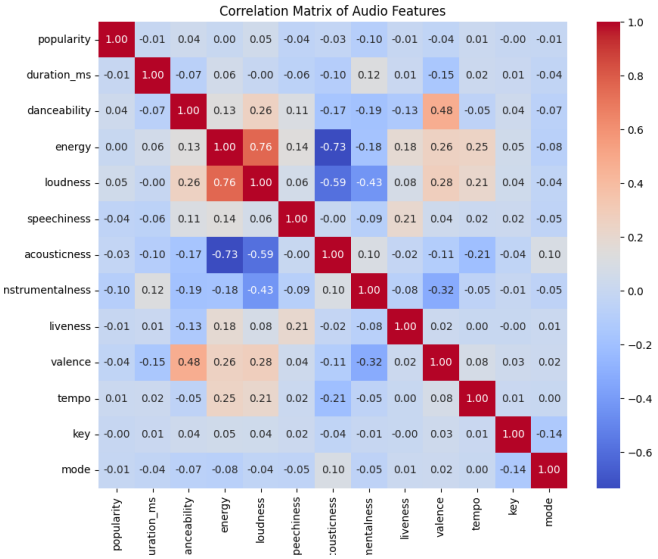

Figure 2. Correlation Matrix of Audio Features.

We can notice some interesting pattern: a positive correlation between `energy` and `loudness` ($r = 0.76$) and between `valence` and `danceability` ($r = 0.49$), and a negative correlation between `acousticness` and both `energy` ($r = -0.73$) and `loudness` ($r = -0.58$). All of these feature correlations make sense with the single feature descriptions. Because these values are moderate, they do not justify the use of feature selection or dimensionality reduction via PCA, as explained better in Section 3.1.

One expected pattern did not emerge: music theory suggests that `mode` (major vs. minor tonality) should affect perceived valence (major tonality creates a brighter and happier sound, while a minor tonality more melancholic sound) and `energy`. No such relationship was observed in the data. Additional patterns and analyses are available in the EDA notebook included in the project repository.

2. Methodologies

As stated before, three different tasks were applied: popularity regression, genre classification, and content-based recommendation. This section will explore the methodologies employed for each task.

2.1. Popularity Regression

The objective of this task is to predict the popularity of a track, defined as an integer score bounded in the range $[0, 100]$, using acoustic features and song metadata.
Intuitively, audio features alone are not expected to fully explain a track's popularity, which is mainly influenced by the artist's fame and other factors external to this dataset, like promotion campaigns, and social media trends. This task acts both as a predictive exercise and as an empirical assessment of what audio features and metadata can capture.

2.1.1. Preprocessing Pipeline

Two distinct preprocessing pipelines were built, one per model family, since linear and tree-based models have different requirements:

- **Tree-based models** (Random Forest, XGBoost): continuous and discrete features are passed through unscaled, since decision trees only rely on the relative order of values rather than their magnitude. High-cardinality categorical features (`track_genre`, and later `artists`) are handled via target encoding.
- **Linear and neural models** (Linear/Ridge/Lasso/ElasticNet, MLP): continuous features are normalized with a Standard Scaler, discrete features with low cardinality (`key`, `time_signature`) are one-hot encoded; `track_genre` is one-hot encoded rather than label or target encoded, since high-cardinality target encoding is more naturally suited to tree splits.

For high-cardinality target-encoding, K-fold cross-fitting with shrinkage toward the global mean (`sklearn.preprocessing.TargetEncoder`, `smooth="auto"`) was used; each row's encoded value is computed by excluding its own fold, preventing data leakage. The train/test split (80/20) was stratified on `track_genre`, used as a proxy since `popularity` itself is continuous. The same stratidfied 5-fold cross validation split waas reused for model comparisions and hyperparameter tuning, to ensure a fair commparision.

2.1.2. Candidate Models

A first round of candidates was trained with default hyperparameters to identify promising model families, including a Dummy Regressor that predicts the mean as a baseline:

- **Dummy Regressor** (baseline)
- **Linear Models**: Linear Regression, Ridge, Lasso, Elastic Net
- **Polynomial Regression** (degree 2, Ridge-regularized)
- **MLP Regressor**
- **Random Forest Regressor**
- **XGBoost Regressor**

Correlation among continuous features was moderate (maximum $r = 0.76$, between `energy` and `loudness`), and Ridge perfomed identically to a plain Linear Regression in this first comparision, idicating that multicollinearity was not a practical problem. Dimensionality reduction (PCA) and feature selection techniques were therfore not applied.

2.1.3. Hyperparameter Tuning

The five candidates yielding the best performances (excluding Linear Regression, which has no hyperparameters to tune) were selected for a randomized hyperparameter search (`RandomizedSearchCV`), using the same 5-fold cross validation splits, optimizing for RMSE.

2.1.4. Hurdle Model

Since the RMSE was not still not satisfiable and a big part of tracks has zero popularity, an hypothesis was made: if we feed the model with only the tracks with popularity greater than zero, the performance will improve, making the task easier by treating the 0-popularity songs as noise. To test this, a two-stage "hurdle" architecture was explored: a binary classifier first predicts wheter a tracks has zero or non-zero popularity, and only the tracks predicted as non-zero are passed to the regressor. Two classifier candidates were compared, Logistic Regression and Random Forest, mirroring the linear/tree model families used before; Random Forest achieved a higher F1 score (0.971) and was selected for the hurdle stage, however this metric should be interpreted with some caution given the class imbalance toward non-zero popularity tracks. The full hurdle pipeline (classifier + regressor) was then compared against the single-stage regression using the same cross-validation protocol; results are discussed in Section 3.

2.1.5. Artist Popularity Feature

As a second refinement, to increase RMSE further, an artist-level feature was introduced to capture popularity signals driven by the fame of an artist, going beyond audio content. Computing an artist's mean popularity across its own tracks would leak the target into the feature (in the extreme case of an artist with a single track, the feature would equal the target exactly). This issue was avoided by applying the same target-encoding scheme (K-fold cross-fitting with shrinkage) used for `track_genre` to the `artists` column: each track's encoded artist-popularity value is computed from the other track of that artist only, within the corresponding training fold, and artists with few tracks are shrunk towards the global mean.

After introducing this feature, the top three models (Random Forest, XGBoost MLP) were re-tuned via a second hyperparameter search undere the same protocol. Final model

selection, quantitative results and error analysis are reported in Section 3.

2.2. Genre Classification

2.2.1. Objective and Challenge

The second task is to predict a track's genre from its audio features. The dataset contains 114 native genre labels. A known challenge that also came out during EDA, is that many genres overlap substantially in audio-feature space and a moderate fraction od tracks are associated with more than one genres (e.g. *chill*, *soul*; or *rock*, *alternative*): approximately 18% of tracks carried multiple tags before the deduplication (using the `track_id`) done during the EDA. This directly caps the achievable performance of any classifier trained on these 114 native labels, regardless of model quality.

2.2.2. Baseline: Native Genre Classification

A baseline light classifier (LightGBM, using the same tree-model preprocessing pipeline as in regression task, with target-encoded `track_genre` excluded from the feature set as it is now the prediction target) was trained on all the 114 genres, evaluated via 5-fold stratified cross validation. As anticipated, this baseline performed close to random guessing, confirming the need for new aggregated labels.

2.2.3. Macro-Genre Construction via Clustering

To address the overlap among genres, an unsupervised clustering step was introduced to group genres into a smaller number of larger macro-genres, used as a relaxed target label. Rather than clustering individual tracks, clustering was performed on genre-level audio profiles: for each of the 114 native genres, the mean of each continuous audio feature was computed (using training data only, to avoid leakage), producing a 114×10 matrix of standardized genre profiles across the continuous acoustic features.

K-Means was applied to this matrix for a range of candidate values of K (4 to 30), and the silhouette score was used as a first selection criterion. The highest silhouette score was obtained at $K = 5$. However, silhouette score alone does not guarantee a balanced or musically coherent partition: inspecting the resulting clusters showed a strong size imbalance, since one cluster contains 60 of 114 genres, and another a single one. To validate better the choice of K, a second comparison was performed in $K \in \{5, 6, 7, 8, 10, 13\}$, considering at the same time, alongside the silhouette score, the cluster size imbalance ratio, and a downstream macro-genre classification F1 (using a lightweight RF classifier). $K = 5$ was confirmed as the best choice: larger values of K reduced cluster imbalance but they did not improve both cluster cohesion and classification performance, indicicating that the residual overlap between some clusters (e.g. between heavy/extreme genres and the pop-rock-indie cluster) is a structural property of the audio feature space rather then an artifact of the chosen K.

2.2.4. Macro-Genre Classification

Using the resulting 5 macro-genre labels, a set of candidates models was evaluated:

- **Logistic Regression**
- **Random Forest**
- **XGBoost**
- **LightGBM**
- **MLP**
- **KNN**

These models are tested under the same 5-fold stratified cross validation protocol, now stratified directly on the macro-genre label. The top 4 candidates by macro-averaged F1 score were then selected for hyperparameter tuning via randomized search. Cohen's Kappa (correcting fot the differerce in chance-level agreement between the 114-class baseline and the 5-class macro-genre task) was used alongside F1 score to validate that improvements were not only due to the reduced label space. Final model selection, metrics, confusion matrix analysis are reported in Section 3.

2.3. Recommendation System

2.3.1. Objective

The third task is to build a content-based recommendation system that, given a track as a query, returns the 10 most acoustically similar tracks from the catalog, reproducing something like the Daily Mix or the Discover Weekly playlists on Spotify. Unlike collaborative filtering, which requires user-item interaction history and is affected by the cold-start problem for new tracks [1], a content-based approach relies only on each track's own audio features, making it applicable to any track in the dataset regardless of its listening history.

2.3.2. Similarity Feature Space

Each track is represented as a vector of scaled continuous audio features combined with one-hot encoded discrete features (`key`, `time_signature`) and binary features (`explicit`, `mode`). For this task, `popularity` and the artist-popularity feature are intentionally excluded from this feature space, since they would dominate the similarity, biasing recommendations toward tracks that are already popular.

Nearest neighbours are retrieved using the cosine similarity (`sklearn.neighbors.NearestNeighbors`), fit on the training split (used as the recommendation catalog), with the test split used to simulate the incoming queries.

2.3.3. Mood Clustering

To make the track-level analysis more complete, K-Means clustering ($K = 4$) was applied to `valence` and `energy`,

following the valence-arousal approach (circumplex) commonly used in Music Emotion Recognition. Restricting the clustering to two dimension allows each cluster to be visualized directly on the valence-arousal plane and assigned an interpretable label (*Happy/Energetic*, *Calm/Content*, *Tense/Angry*, *Sad/Melancholic*), based on the sign of each centroid's cluster relative to the midpoint of both axes. This mood label is presented as an optional feature alongside recommendations, distinct from the audio similarity ranking itself.

2.3.4. Macro-genre Filtering

The recommender optionally reuses the $K = 5$ macro-genre mapping obtained in the classification task as a filter: candidates can be restricted to the query's own macro-genre before ranking by similarity. This design choice ties together the three tasks of this project.

2.3.5. Hidden Gems Mode

The regression stage also plays a role in this task. Specifically, the tuned regression model from Section 3.1 is used to compute, for every catalog track, a *hidden-gem score*: the difference between the model's predicted popularity and the track's actual observed popularity. A large positive score indicates a track the model expected to be more popular than it actually is, given its audio, artist, and genre profile, with the goal of making the user discver more underrated tracks. Re-ranking candidates with this score, instead of similarity alone, offers an alternative recommendation mode oriented toward discovery rather than pure acoustic resemblance.

2.3.6. Evaluation

Since there is no ground-truth user-feedback for this task, three proxy metrics were used:

- **Genre-agreement@10**: the fraction of the top-10 recommendations sharing the query genre, computed both on native genres and on the $K = 5$ macro-genre.
- **Popularity bias check**: the average popularity of recommended tracks compared to the catalog average, computed separately for standard and hidden-gems recommendations.
- **Qualitiative case studies**: a small number of example queries, inspected manually across recommendation modes.

3. Results

3.1. Popularity Regression

Table 4 summarized the first comparison across candidate models with default hyperparameters. Random Forest, MLP, and Polynomial+Ridge performed best, while Lasso and Elastic Net remained close to the baseline, indicating that default regularization was too aggressive for this set of features.

Table 4. Stage 1 model comparison (default hyperparameters, 5-fold CV).

Model	**RMSE**	R^2
Dummy (mean)	19.394	-0.000
Linear Regression	14.854	0.413
Ridge	14.854	0.413
Polynomial + Ridge	14.801	0.417
Lasso	19.006	0.040
Elastic Net	19.002	0.040
Random Forest	14.623	0.431
XGBoost	14.861	0.413
MLP	14.525	0.439

Correlation among continuous audio features was moderate (Section 1.4), and Ridge performed virtually identically to plain Linear Regression, indicating multicollinearity was not a practical issue; PCA was therefore not applied.

A hurdle-model architecture (binary zero/non-zero classifier followed by regression on the non-zero subset, Section 2.1.4) was compared against single-stage regression on the same top candidates. As shown in Table 5, the hurdle approach shows no significative performance advantage. Tracks with zero popularity tracks are better represented with the same continuous distribution rather than a separate class, so a single-stage regression was preferred.

Table 5. Hurdle model vs. single-stage regression (5-fold CV RMSE).

Model	**Single-stage**	**Hurdle**
MLP	14.479	15.431 ± 0.161
Random Forest	14.503	15.417 ± 0.119
XGBoost	14.565	15.599 ± 0.097

Adding a target-encoded artist-popularity feature (Section 2.1.5) produced the largest single improvement observed in this task (Table 6), confirming that artist identity carries substantial signal beyond audio content alone.

Table 6. Effect of adding the artist-popularity feature (5-fold CV).

Model	**RMSE**	R^2
Ridge + artist	13.519	0.514
Polynomial + Ridge + artist	13.484	0.517
MLP + artist	13.142	0.541
Random Forest + artist	13.090	0.544

After a second hyperparameter search on the top three candidates, Random Forest achieved the best validation RMSE (12.980), slightly ahead of XGBoost (13.030) and

MLP (13.131), and was selected as the final model, not only for the performance, but also because its feature importance make the predictions much more easier to interpret. Table 7 reports its performance on training, cross-validation, and on the test set.

Table 7. Final model (Random Forest + artist, tuned) performance.

Set	RMSE	MAE	R^2
Train	7.696	4.501	0.843
CV (5-fold)	12.980	8.436	0.552
Test	12.995	8.326	0.555

The consistent results across CV and test sets indicate that tuning did not overfit the validation data. In addition, the gap between the train and the test results, is a common characteristic of deep Random Forest models.

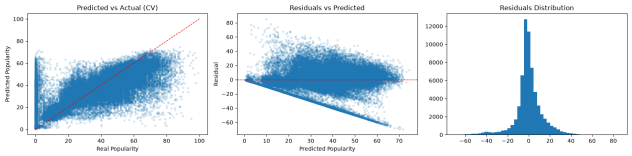

Figure 3. Predicted vs. actual popularity, residuals vs. predicted, and residual distribution (training set, cross-validated predictions).

Figure 3 reveals that the model struggles at the extremes, underestimating highly viral songs and showing high variance for zero-popularity tracks. Looking at feature importances (Figure 4), the explanation is clear: target-encoded artist and genre signals dominate the model, carrying far more weight than raw acoustic features. (Figure 4) confirms that artist and genre target-encoded features dominate, far ahead of any individual audio feature.

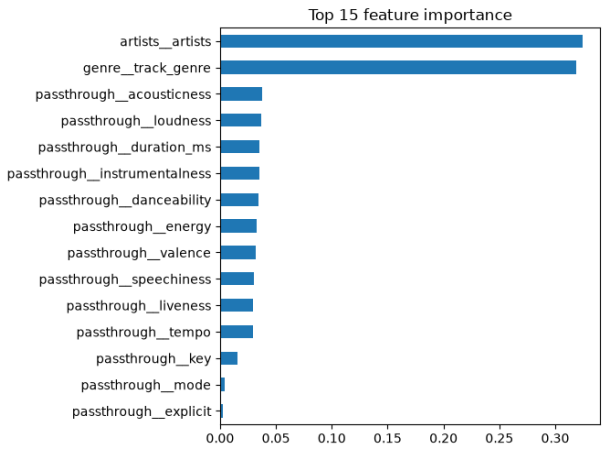

Figure 4. Top 15 feature importances, final Random Forest model.

Error analysis confirms that prediction residuals scale directly with the internal popularity variance of an artist's catalog (Figure 5): artists with both hits and flops are where the model struggles most, since no available feature distinguishes *which* specific track will succeed within the same artist/genre.

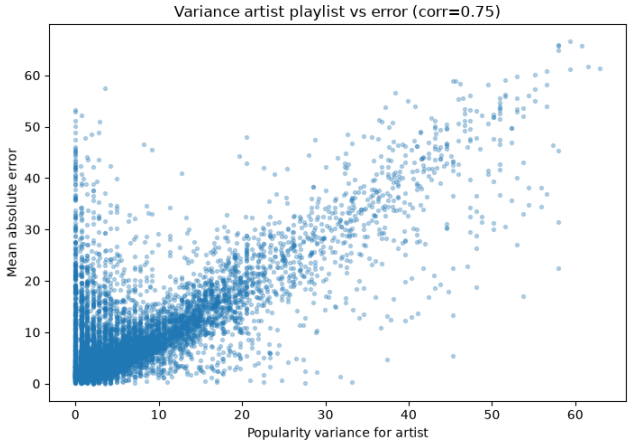

Figure 5. Artist catalogue popularity variance vs. mean absolute prediction error.

3.2. Genre Classification

The baseline classifier trained on all 114 native genes achieved a macto F1 of 0.067 an Cohen's Kappa of 0.067, confirming the overlap already suggested by the EDA.

Figure 6 shows the silhouette score across $K = 4$ to 30 for K-Means clustering of genre-level audio profiles, peaking at $K = 5$. Since silhouette alone does not take into account the cluster balance, $K = 5$ was further validates against a range of alternatives (Table 8), confirming it as the best compromise despite its size imbalance.

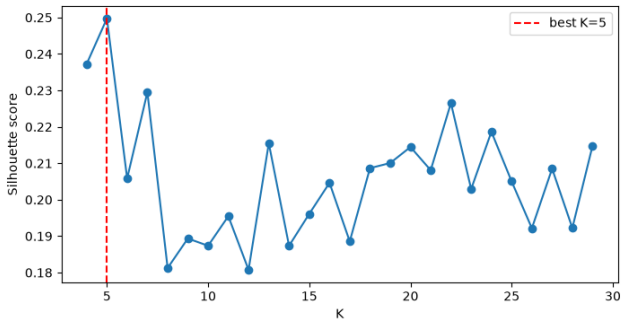

Figure 6. Silhouette score across candidate values of K.

Table 9 compares candidate models on the resulting 5-macro-genre classification task; Table 10 reports results after hyperparameter tuning of the top four candidates.

XGBoost was selected as the final model. Table 11 summarizes the improvement over the native-genre baseline, us-

Table 8. Multi-criteria comparison of candidate K values.

K	Silhouette	Imbalance	F1
5	0.250	40.4	0.770
6	0.206	25.7	0.726
7	0.230	29.2	0.695
8	0.181	19.8	0.658
10	0.187	18.3	0.684
13	0.215	19.6	0.645

Table 9. Stage 1 macro-genre classification (default hyperparameters).

Model	F1 macro
XGBoost	0.786
Random Forest	0.778
MLP	0.759
LightGBM	0.754
KNN	0.735
Logistic Regression	0.591

Table 10. Macro-genre classification after hyperparameter tuning (5-fold CV).

Model	F1 macro
XGBoost	0.795
LightGBM	0.792
Random Forest	0.779
MLP	0.768

ing Cohen's Kappa to correct for the difference in chance-level agreement between a 114-class and a 5-class task.

Table 11. Native-genre baseline vs. final macro-genre model (test set).

Setup	F1 macro	Cohen's κ
Baseline (114 native genres)	0.067	0.067
Macro-genre ($K = 5$)	0.797	0.661

Errors concentrate almost entirely between two macro-genres (Figure 7), suggesting that some sub-genres (e.g. punk-rock, hard-rock) are acoustically closer to mainstream pop/rock than to the rest of their assigned macro-genre, a limitation of clustering rather than of the classifier itself.

3.3. Recommendation System

Figure 9 shows the four mood clusters obtained from valence-energy K-Means, together with their interpreted labels.

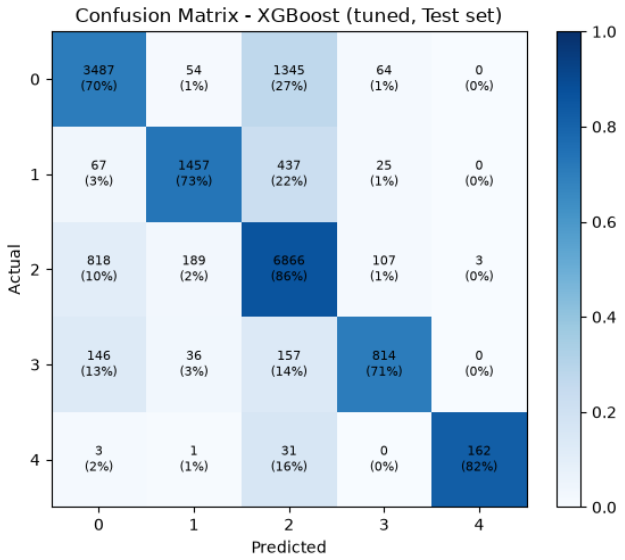

Figure 7. Normalized confusion matrix, macro-genre classification (test set).

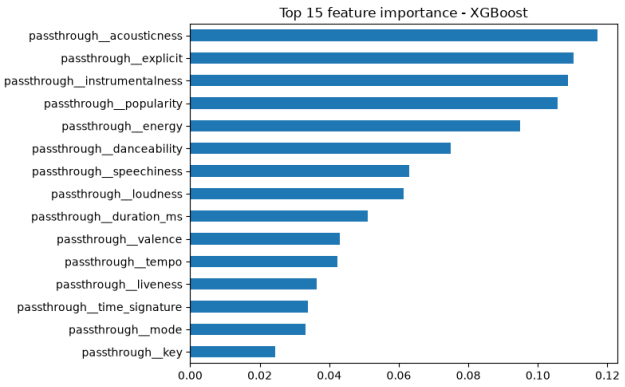

Figure 8. Top 15 feature importances, final macro-genre classifier.

Table 12 reports the evaluation metrics described in Section 2.3.6, averaged over 200 random test-set queries.

Table 12. Recommendation system evaluation metrics.

Metric	Value
Genre-agreement@10 (native, 114 classes)	12.7%
Genre-agreement@10 (macro, 5 classes)	62.0%
Avg. popularity — catalogue	35.25
Avg. popularity — standard recommendations	35.11
Avg. popularity — hidden-gems recommendations	18.62

Agreement rates are consistent with the classification results in Section 3.2: native genre agreement is just at 12.7%, whereas macro-genre agreement climbs at 62.0%. While baseline recommendations exhibit negligible popu-

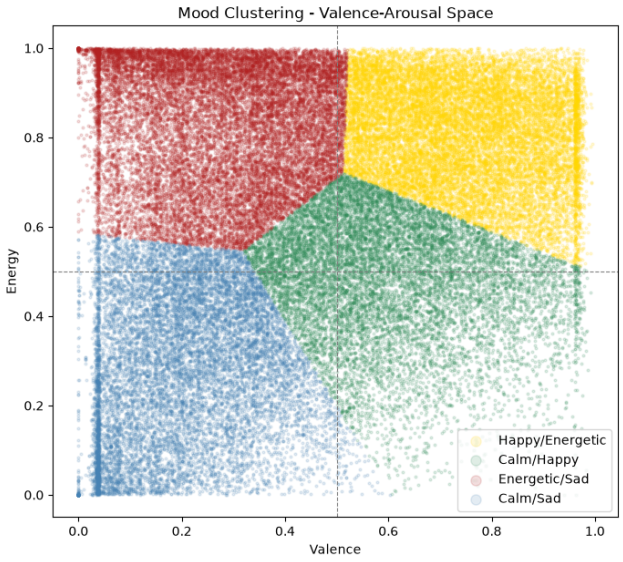

Figure 9. Mood clusters on the valence-arousal plane.

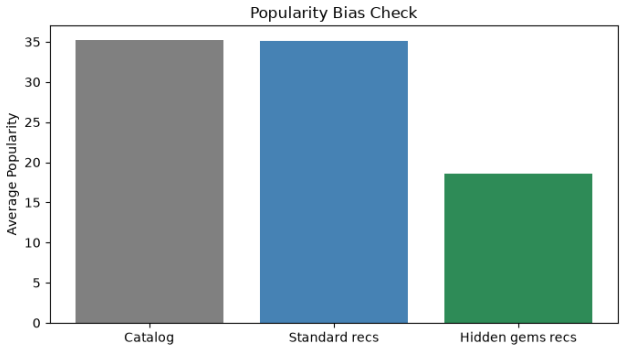

Figure 10. Average popularity: catalogue vs. standard vs. hidden-gems recommendations.

larity bias, the hidden-gems variant yields significantly less popular tracks, expect also by the regression model underprediction of viral hits (Section 3.1).

4. Conclusions

This study connected three machine learning tasks on the Spotify dataset: popularity regression, genre classification and song recommendation. Rather than treating these problems isolated, the main priority was to understand why acoustic features succeed or fail to explain some musical outcomes, using the lessons learned in each task to guide the design of the next one.

4.1. Summary of Findings

Overall, the findings of this project show a clear pattern: audio features alone simply cannot predict commercial success. However, they become useful once we consider artist context and group genres by shared acoustic traits. Across the three task, the experiments highlighted some takeaways:

- **Popularity prediction:** Raw audio traits explain only a modest fraction of track popularity ($R^2 \approx 0.41$–0.44). Adding target-encoded artist and information about genre provided the single largest performance improvement ($R^2 \approx 0.55$), confirming that commercial success depends far more on who makes the song rather than on acoustic profile alone. Testing a hurdle architecture revealed that zero-popularity songs are not a structurally distinct group, but rather the flat bottom of a continuous spectrum. Error analysis confirmed that the models struggle most with artists with both major hits and less popular tracks: it recognizes a famous name, but lacking marketing data, it has no way to guess which particular song will blow up.
- **Genre classification:** Trying to tell 114 genres apart just from audio traits was practically impossible ($\kappa = 0.067$), mostly because many styles sound alike and songs often fit into multiple categories. Grouping them into five macro-genres finally cleared up the chaos ($\kappa = 0.661$). Even the mistakes that remained made sense, mostly happening between genres that genuinely sound alike, like hard rock blurring into pop/rock.
- **Content-based recommendation:** The recommender showed a similar pattern from a listener's point of view. Recommended tracks matched the original genre only 12.7% of the time, but aligned on macro-genres 62.0% of the time, just like in classification. Leaving popularity out of the similarity search kept the system from just pushing the most famous songs. At the same time, the 'hidden-gems' option worked well to bring lesser-known tracks to the surface, essentially turning the regression model's habit of underpredicting huge hits into a useful feature.

4.2. Limitations

Some structural limitations were observed during this project:

- **Missing external context:** The missing some very informative features such as playlist placement, marketing campaign, release date, and social media reach, factors that are intuitively responsible for most unexplained variance mainly across the popularity regression task.
- **Deduplication bias:** Forcing a track with different genre tags to have a single genre label causes an artificial limit on classification.
- **Acoustic oversimplification:** Because macro-genre clustering groups genres by their average profile, it overlooks variation among songs in the same genre and naturally produces unbalanced clusters.
- **Cold start and diversity:** The recommender was not tested on brand-new artists. Furthermore, while the hidden-gems mode addresses popularity bias, it does not

guarantee variety among artists, meaning that a prolific artist with similar tracks can still dominate the suggested list.

4.3. Future Work

Several directions could extend this work. Adding external context like release dates and playlist inclusions would help address the performance cap obtained in the regression analysis. For classification, it may be interesting to explore different approaches to cluster the genres, like soft or hierarchical clustering, to solve the genre overlapping and cluster imbalance. Finally, encouraging more artist variety would keep the recommender from leaning too heavily on the same few names.

References

[1] Yehuda Koren, Robert Bell, and Chris Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009. 2, 5

[2] Maharshi Pandya. spotify-tracks-dataset, 2026. 1

[3] Mohamed Sordo and et al. Music information retrieval: A survey. *Journal of Music Information Retrieval*, 2012. 2